

EGCO 425 – Chapter 3

Data Cleansing

Data Transformation

Feature Selection & Aggregation

References

- [1] Tan et al., Introduction to Data Mining (Chapter 2-3)
- [2] Chisholm, Exploring data with RapidMiner (Chapters 4, 6, 8)
- [3] Kotu & Deshpande, Data Science: Concepts and Practice (Chapter 14)
- [4] RapidMiner Doc, <https://docs.rapidminer.com/latest/studio/operators/>

Data Preprocessing

Issues from data exploration	Preprocessing
Size of data • Too many records • Too few records, class imbalance • Too many attributes	• Sampling without replacement • Sampling with replacement • Selection, aggregation
Attribute values • Typos, invalid values • Different ranges of attributes • Noises, outliers • Missing values	• Correction, elimination • Normalization • Elimination, robust methods • Elimination, imputation
Attribute types • Improper types	• Type conversion
Multiple data sets • New records • New attributes	• Appending • Joining

Chapter Outline

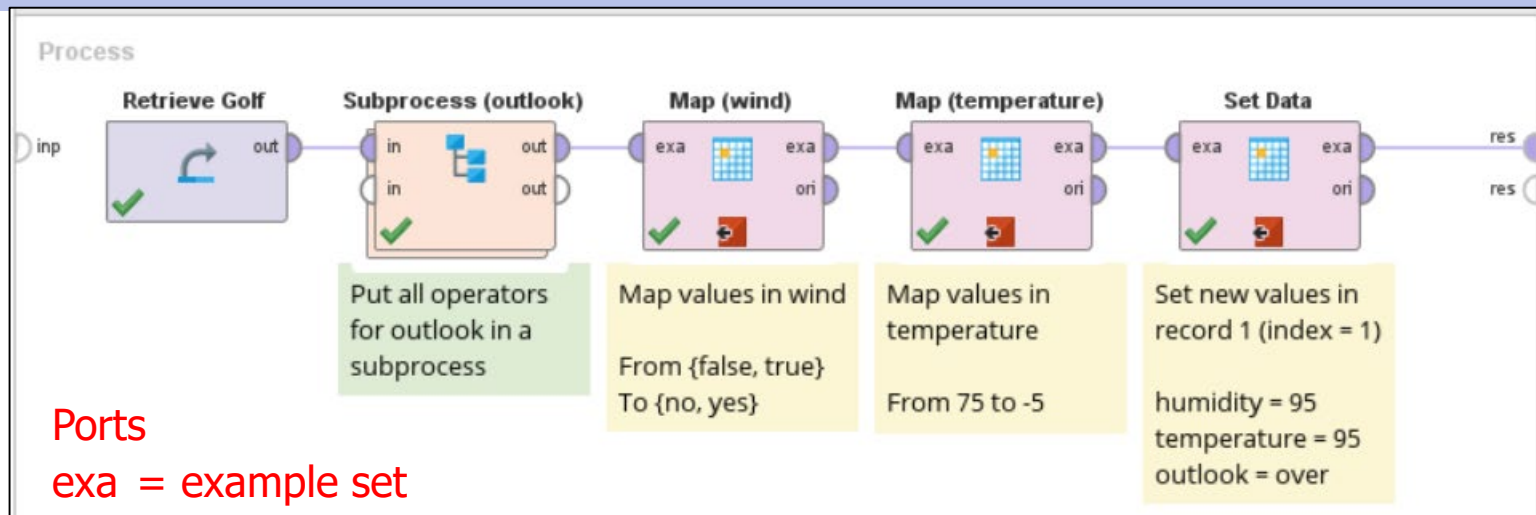
- ⊕ Preprocessing attributes
 - ▶ Value manipulation
 - ▶ Normalization
 - ▶ Type conversion
- ⊕ Handling potential problems
 - ▶ Noises and outliers
 - ▶ Missing values
- ⊕ Preprocessing records & tables
 - ▶ Sampling & record manipulation
 - ▶ Table manipulation
- ⊕ Feature selection & aggregation
 - ▶ Attribute scoring, subset selection
 - ▶ Principal component analysis (PCA)

Value Manipulation in RapidMiner

Nominal values	
Cut	New value = substring of old value
Merge	New value = { old value1, old value2, ... }
Replace	Replace old value with a new value, according to conditions
Trim	Remove leading & trailing spaces from old values

Both nominal & numeric values	
Map	Replace old values with new ones Can do only 1 mapping for 1 type of attribute at a time
Set	Set new values in a specified record

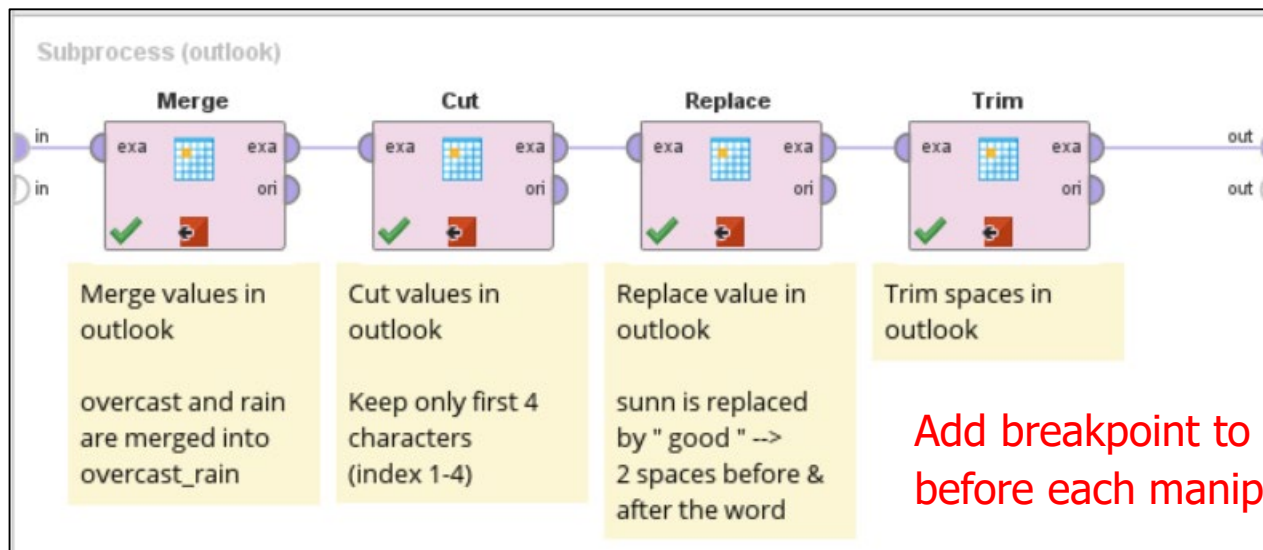
Example (1) : value manipulation



Ports

exa = example set

ori = original example set



Folder
Blending.Value

Add breakpoint to see data before each manipulation

Normalization

- ⊕ Some methods typically work with certain range of data e.g. some neural network implementation expect values in $[0, 1]$ range
- ⊕ Different ranges of attribute values (e.g. age and salary) affect distance-based methods such as KNN, K-means clustering
- ⊕ Normalization transform **numeric attribute** by changing its scale or range to a new scale

Normalization Methods

⊕ Min-max normalization

- ▶ Transform old values (in old range) to get new values (in new range) and the relationship among the old values is preserved

$$newValue = (oldValue - oldmin) \times \frac{newrange}{oldrange} + newmin$$

⊕ Decimal scale normalization

- ▶ E.g. range of data = [-986, 917]
- ▶ Divide each value by 1000, new range = [-0.986, 0.917]

$$newValue = \frac{oldValue}{d} \quad ; \text{ where } d = 10, 100, 1000, \dots$$

⊕ Standardization (Z-score normalization)

- ▶ New values have mean = 0, std. deviation = 1

$$newValue = \frac{oldValue - \bar{x}}{s_x}$$

⊕ Interquartile range normalization

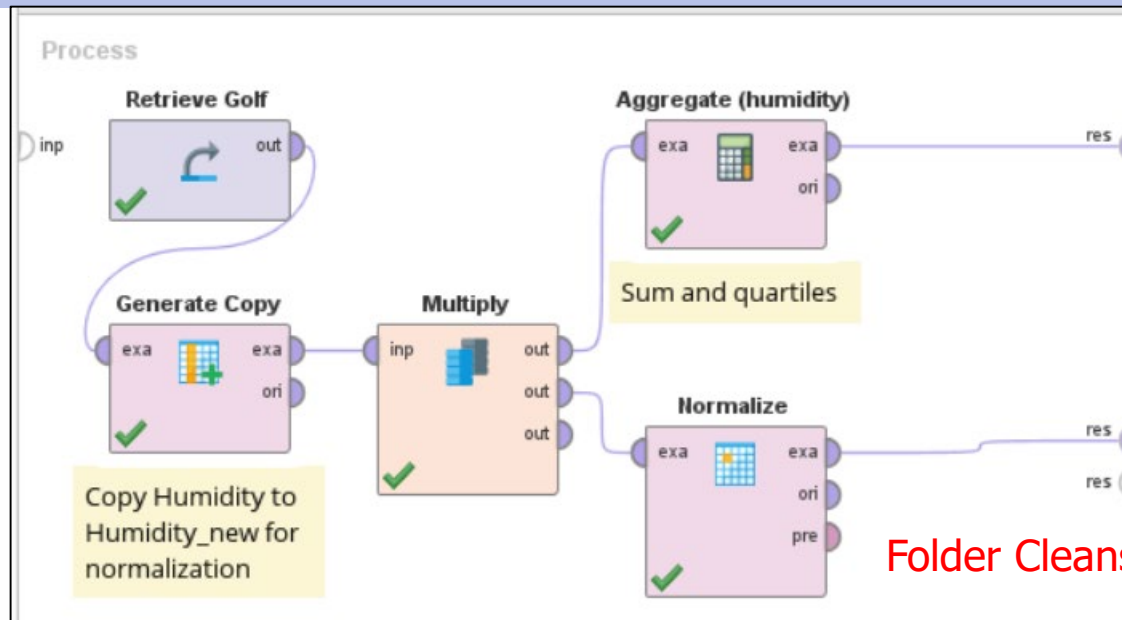
- ▶ Robust to outliers (extreme values beyond Q1 and Q3 are excluded)

$$newValue = \frac{oldValue - median}{(IQR/1.349)} \quad IQR \approx 1.349 \text{ times of } SD$$

⊕ Proportion normalization

$$newValue = \frac{oldValue}{Sum \text{ of all attribute values}}$$

Example (2) : normalization



Sum = 1124

Q1 = P25 = 70

Q2 = P50 = 80

Q3 = P75 = 90

Row No.	Play	Humidity	Humidity_new
1	no	85	0.479
2	no	90	0.987
3	yes	78	-0.232
4	yes	96	1.597
5	yes	80	-0.029

Z-score

Humidity_new
0.645
0.806
0.419
1
0.484

0-1 range

Humidity_new
0.076
0.080
0.069
0.085
0.071

Proportion

Humidity_new
0.337
0.674
-0.135
1.079
0

IQR range

Attribute Type Conversion

⊕ Some type conversion

- ▶ Nominal → binary
- ▶ Nominal, binary → numeric
- ▶ Numeric → nominal, binary

RapidMiner uses
one-hot approach



⊕ Nominal to binary (binarization)

- ▶ Binary encoding = map each category to unique binary code
- ▶ One-hot encoding = map each category to each binary attribute

Old attribute = position
Old values

1 = programmer
2 = system analyst
3 = network admin.
4 = technician
5 = project manager

Binary Encoding		
x1	x2	x3
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0

One-hot Encoding (1:1 map)				
x1	x2	x3	x4	x5
1	0	0	0	0
0	1	0	0	0
0	0	1	0	0
0	0	0	1	0
0	0	0	0	1

⊕ Nominal or binary to numeric

- ▶ Approach 1 : binarize attribute first, then map binary values to {0, 1}

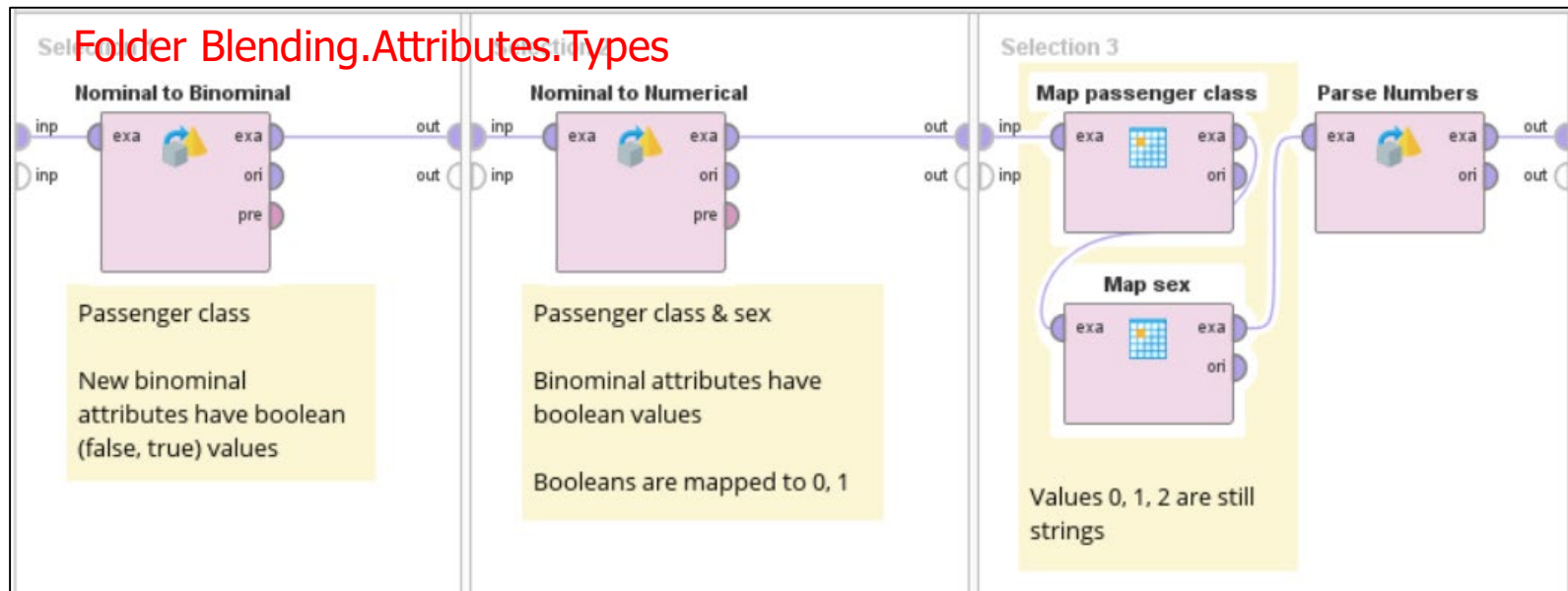
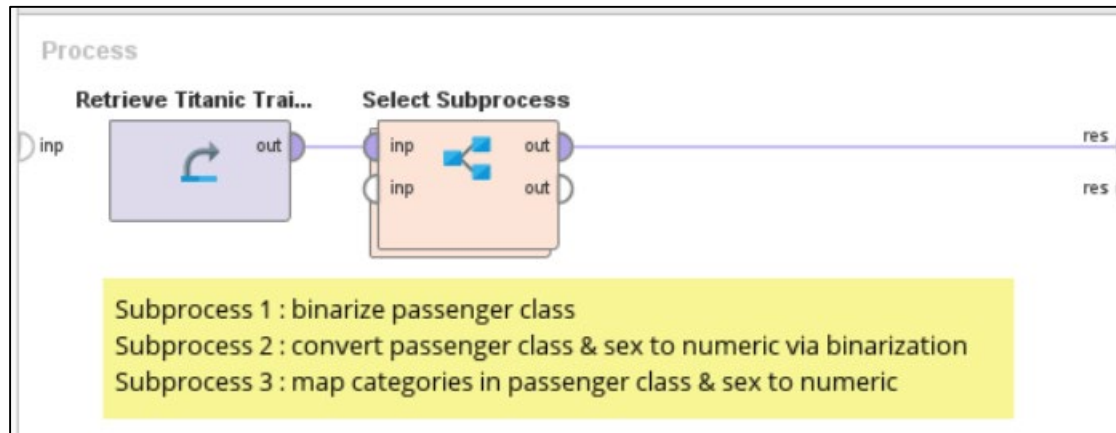
Original attributes	Passenger class = {first, second, third}	Sex = {female, male}
Binarization & binary mapping	Passenger class = first {0, 1}	Sex = female {0, 1}
	Passenger class = second {0, 1}	Sex = male {0, 1}
	Passenger class = third {0, 1}	

- More appropriate because original proportions are preserved
- RapidMiner uses this approach

- ▶ Approach 2 : map categories to numeric values

Original attributes	Passenger class = {first, second, third}	Sex = {female, male}
Category mapping	Passenger class = {0, 1, 2}	Sex = {0, 1}

Example (3) : binarization & numeric conversion



⊕ What are the differences ?

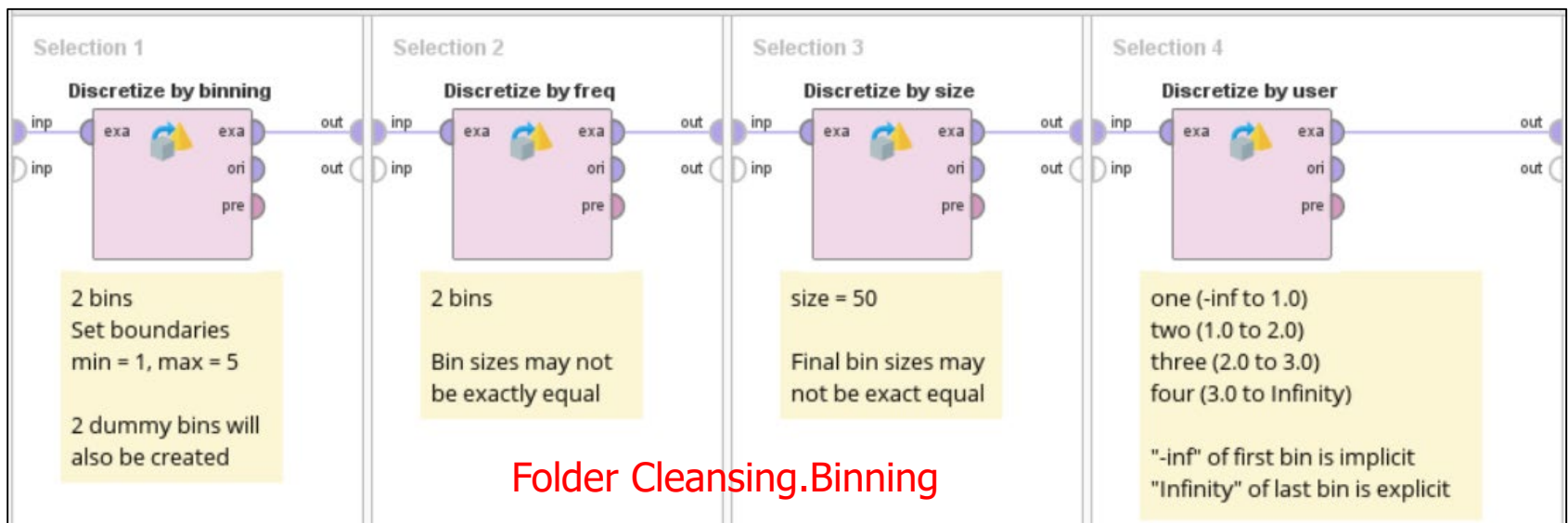
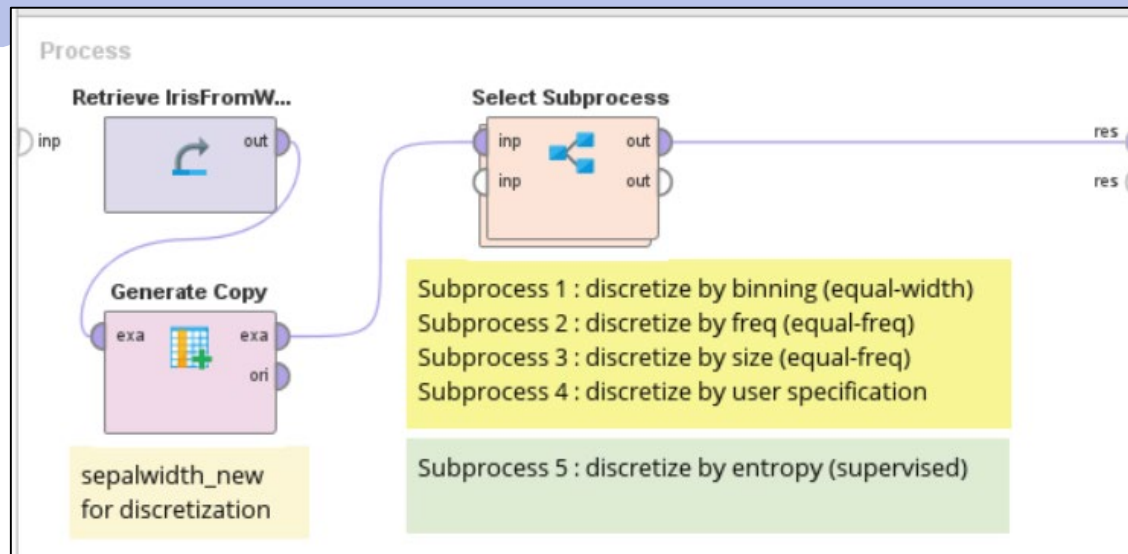
- ▶ Original data 594/916 male (0.648) + 322/916 female (0.352)
- ▶ Subprocess 2 generate 2 sex attributes
sex = female (mean = 0.352)
sex = male (mean = 0.648)
- ▶ Subprocess 3 keep only 1 sex attribute, mean = 0.648
- ▶ Subprocess 2 preserves both original proportions, while subprocess 3 loses information about female proportion

Discretization (unsupervised)

- ⊕ Partition data into **bins**
- ⊕ **Equal-width**, i.e. discretization by binning
 - ▶ Given #bins, create bins of equal range
 - ▶ #values in different bins may vary
 - ▶ Suitable for uniform distribution, but can be affected by outliers
- ⊕ **Equal-frequency**
 - ▶ By frequency : given #bins, create bins of equal frequency
 - ▶ By size : given size, create bins of equal size (= freq) as specified
 - ▶ Ranges of different bins may vary
 - ▶ Each bin contain equal amount of data to learn patterns

Record	Old value [0 – 1]	Required bins = 4		Required size = 5
		Equal-width (range = 0.25)	Equal-frequency (freq = 4)	Equal-frequency (#bins = 3)
1	0.15	1	1	1
2	0.18	1	1	1
3	0.20	1	1	1
4	0.20	1	1	1
5	0.22	1	2	1
6	0.25	1	2	2
7	0.30	2	2	2
8	0.33	2	2	2
9	0.44	2	3	2
10	0.46	2	3	2
11	0.52	3	3	3
12	0.76	4	3	3
13	0.79	4	4	3
14	0.81	4	4	3
15	0.92	4	4	3
16	1.00	4	4	3

Example (4.1) : unsupervised discretization



Discretization (supervised)

- ⊕ Entropy-based method. Use class label to supervise discretization
 - ▶ Recursively adding split points to partition data into bins, such that entropy is minimized or information gain is maximized
- ⊕ **Entropy** = measurement of information purity
 - ▶ High information (purity) → low entropy (energy)
 - ▶ In pure bin, all records are from only 1 class → zero entropy
 - ▶ Entropy of each bin = $e(p_1, p_2, \dots) = -p_1 \log p_1 - p_2 \log p_2 - \dots$
 - Use logarithm of base 2 (unit of measurement is bits)
 - $p_1, p_2, \dots$ = class proportions in the bin; they are summed to 1
- ⊕ **Information gain** = entropy before split – entropy after split
 - ▶ If the partition is good, we should get lower entropy & thus higher information gain

Example: entropy calculation

- Sort values in increasing order

Value	64	65	68	69	70	71	72	72	75	75	80	81	83	85
Class	y	n	y	y	y	n	n	y	y	y	n	y	y	n

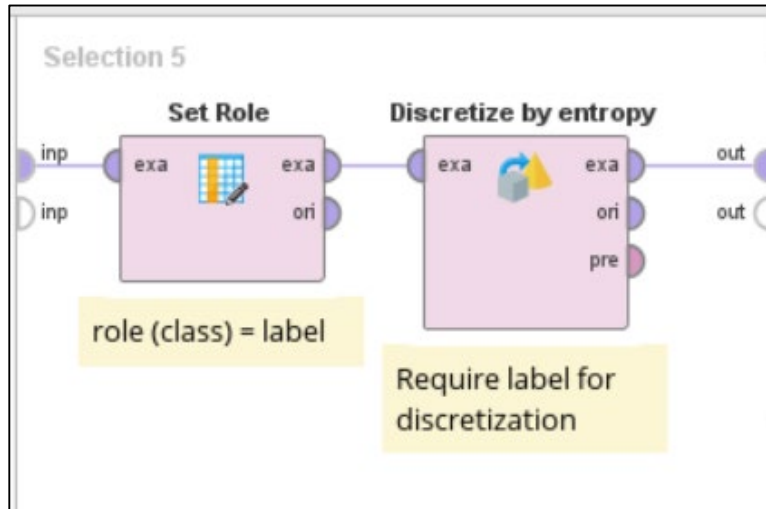
- Consider split point = 71.5

- Entropy before split = $e(\text{proportion of } y, \text{proportion of } n)$
 $= -9/14 \log(9/14) - 5/14 \log(5/14) = 0.9399$

- Entropy after split
 Bin 1 (4 y's + 2 n's) = $-4/6 \log(4/6) - 2/6 \log(2/6) = 0.9183$
 Bin 2 (5 y's + 3 n's) = $-5/8 \log(5/8) - 3/8 \log(3/8) = 0.9544$
 Total entropy = $6/14 e(\text{bin 1}) + 8/14 e(\text{bin 2}) = 0.9389$

Bin sizes are not equal, so entropies are weighted by their relative sizes

Example (4.2) : supervised discretization



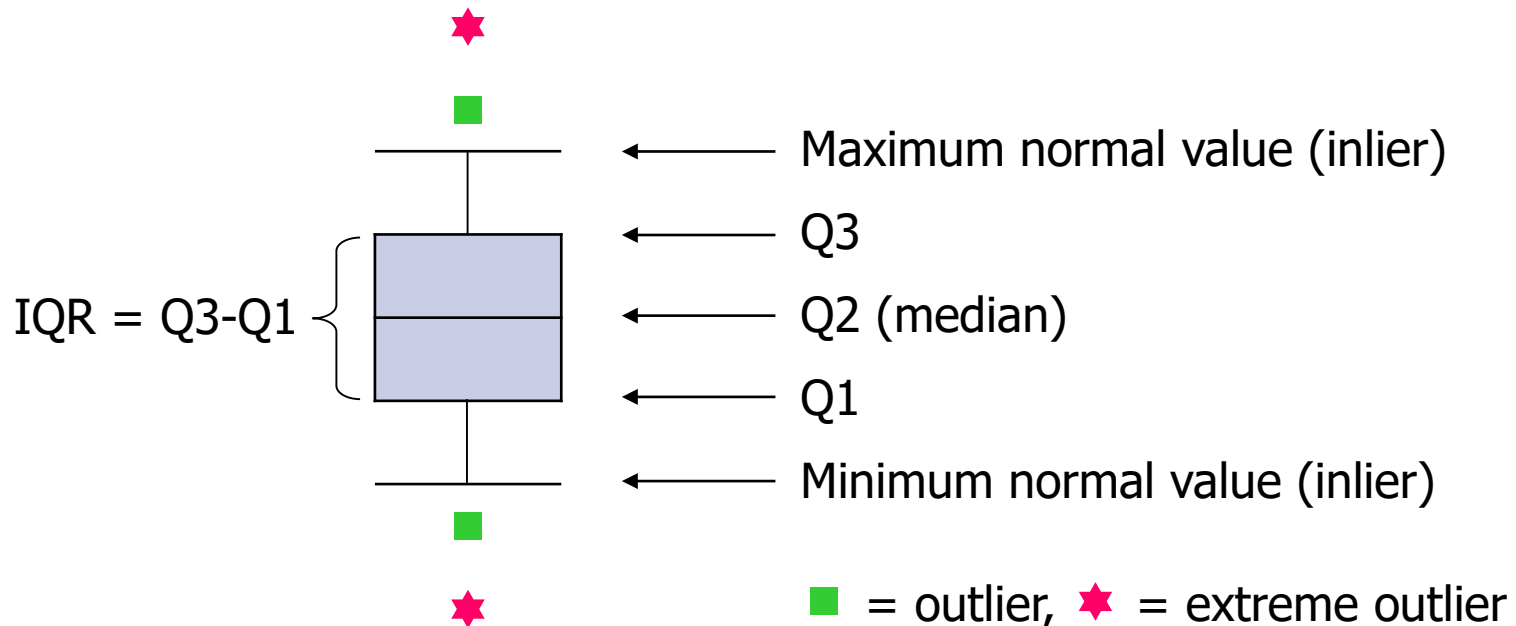
⊕ When to use entropy-based method ?

- ▶ To get meaningful splits → each class has proper amount of data for a classifier to learn the pattern of that class
- ▶ It is called internally by some data mining methods

Noises and Outliers

- ⊕ Noises = distortion of values or addition of unauthentic values
 - ▶ Such values may be unusual or extreme (i.e. outliers) or intermix with authentic values (e.g. random errors, misspellings, contamination)
 - ▶ In spatial or temporal data (e.g. signals), noises are very common
- ⊕ Outliers = unusual out-of-range data
 - ▶ Can be unintentional (e.g. honest errors) or intentional (e.g. fraud credit card transactions)
- ⊕ Detecting and handling noises & outliers
 - ▶ Remove them before applying data analysis or choose data analysis methods that are robust to noises/outliers
 - ▶ Many detection methods are related to clustering
 - ▶ For now, a simple outlier detection method is observing boxplot

Boxplot (inter-quartile range method)



- ⊕ Outliers = values below $Q1 - 1.5 \text{ IQR}$ or above $Q3 + 1.5 \text{ IQR}$
- ⊕ Extreme outliers = values below $Q1 - 3.0 \text{ IQR}$ or above $Q3 + 3.0 \text{ IQR}$

Data Smoothing

For noisy data

⊕ Binning method → put close values in the same bin. Then replace real values with mean or median of that bin

- ▶ Example : given sorted data
4, 8, 15,
21, 21, 24,
25, 28, 34

Partition into (equal-frequency) bins:

Bin 1: 4, 8, 15

Bin 2: 21, 21, 24

Bin 3: 25, 28, 34

Smoothing by bin means:

Bin 1: 9, 9, 9

Bin 2: 22, 22, 22

Bin 3: 29, 29, 29

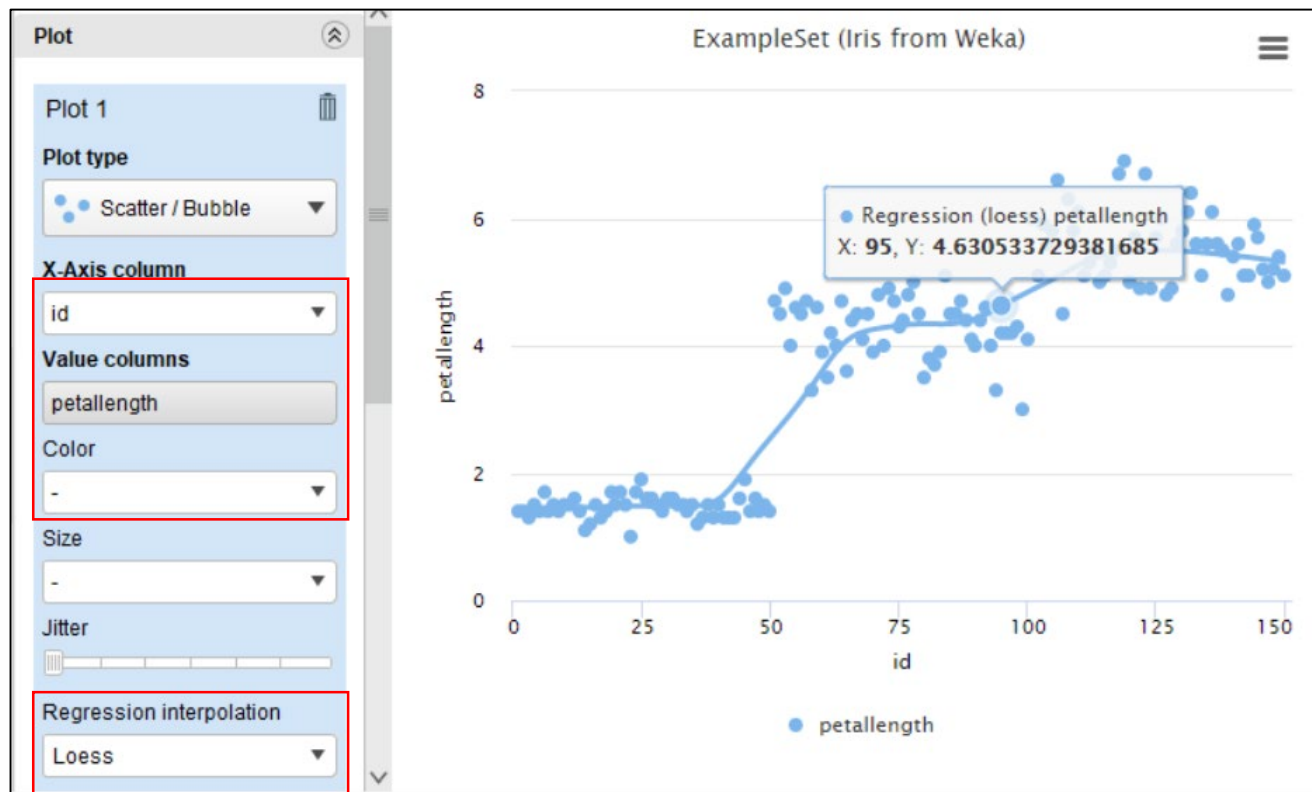
Smoothing by bin boundaries:

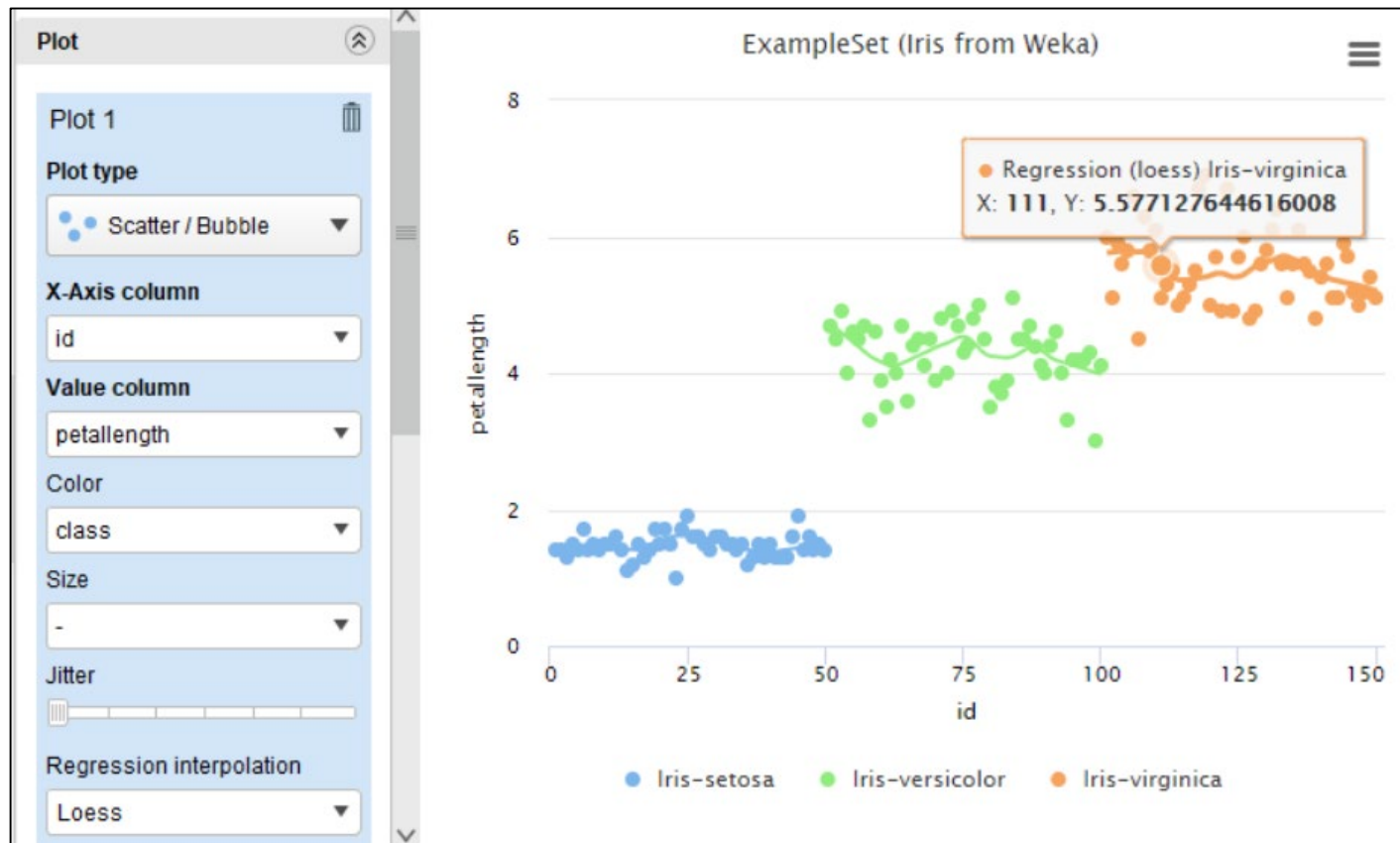
Bin 1: 4, 4, 15

Bin 2: 21, 21, 24

Bin 3: 25, 25, 34

- ✦ Regression method → LOESS (locally estimated scatterplot smoothing)
 - ▶ Fit regression lines on subsets of data (i.e. sliding window) and use values on regression lines instead of actual values





Missing Data

⊕ Types of missing data

- ▶ Consider attribute X with missing values and another attribute Y
- ▶ **Missing completely at random (MCAR)**
- ▶ **Missing at random (MAR)** : missing values in X are independent of each other, but depend on the values in Y
 - E.g. X = sales amount, Y = type of vendor
 - Many missing sales amounts are related to online vendors
- ▶ **Missing not at random (MNAR)** : missing values in X are related
 - E.g. X = average CPU load in each minute
 - Many values are missing because the CPU is very busy during a certain time interval

- ⊕ In many data analysis software including Weka & RapidMiner, missing values are set as "?"
 - ▶ But sometimes, they may be set as special values or categories such as "0", "none", "null"
 - ▶ May be included in the analysis → treated as another category
 - ▶ Or handled as missing values → elimination, imputation

Handling missing values

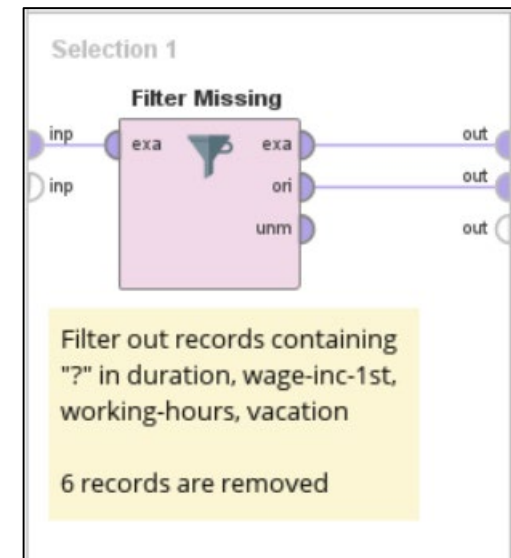
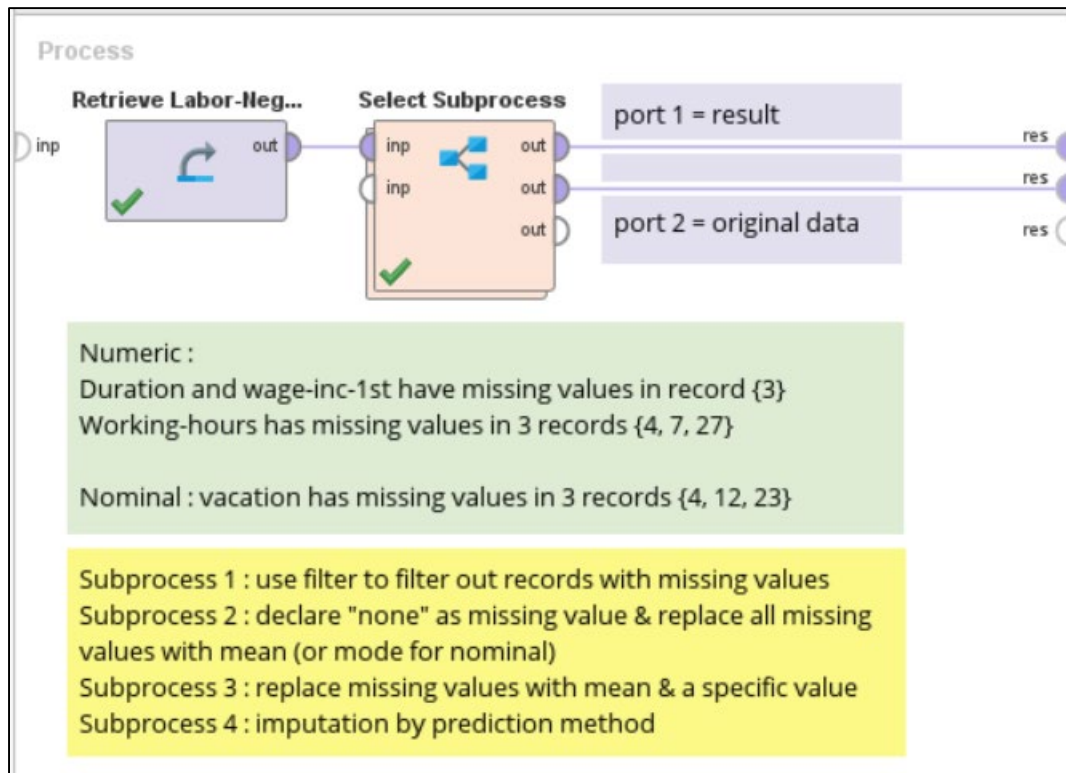
⊕ **Elimination**

- ▶ Remove records that contains missing values
- ▶ Remove attributes that have missing values
- ▶ May end up with too few records/attributes for the analysis
- ▶ Valid for MCAR
- ▶ Some cautions for MAR and MNAR
 - Removing data may cause bias or failure to detect some patterns
e.g. it may appear that the CPU is not loaded rather than heavy loaded
 - Instead, we can predict missing values from other attributes or nearby values in the same attribute

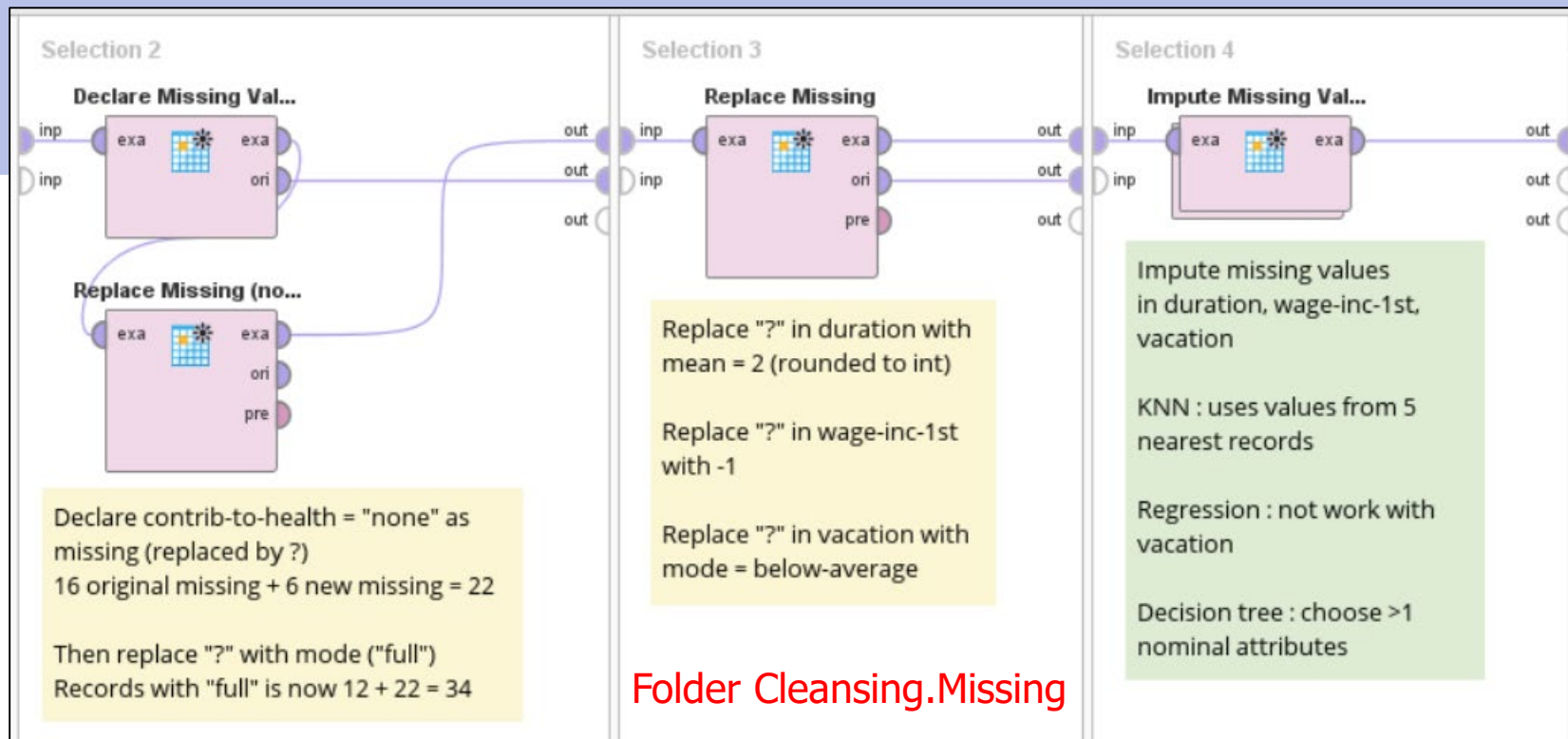
⌘ Imputation

- ▶ Estimate missing values
 - Central tendency (mean, median, mode) of the whole data set
 - Central tendency (mean, median, mode) of the given class
 - Neighboring values e.g. temperatures of a few days before-after the date of missing data, rainfall in nearby districts
 - Prediction using regression or classification model
 - Hot deck imputation, i.e. use values from the most similar objects
- ▶ Preserve all data for the analysis, but improper imputation may cause bias or distortion

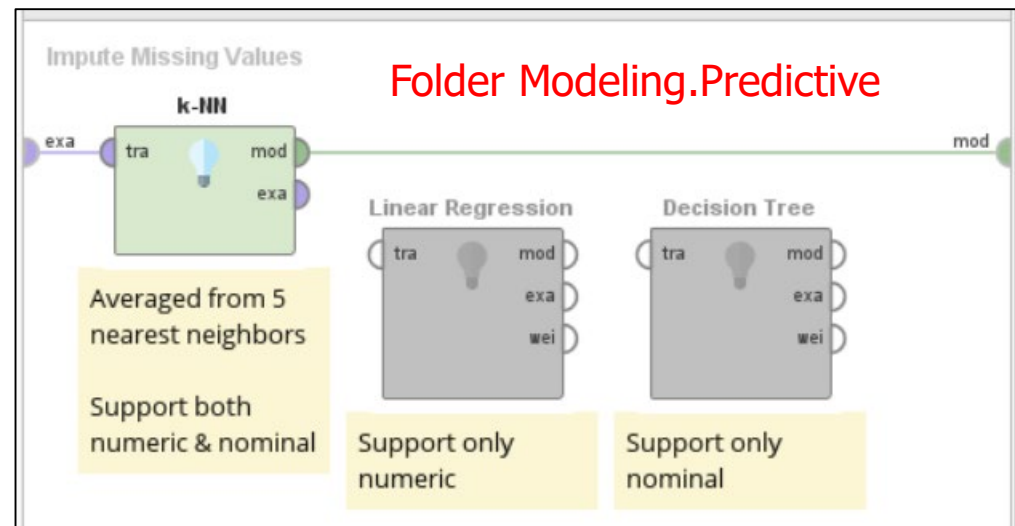
Example (5) : missing values



Folder Cleansing.Missing



Add imputation method inside the subprocess (use classification or regression)



Sampling

- ⌕ **Sampling without replacement** : when an item is selected, it is removed from the population
- ⌕ **Sampling with replacement** : items are not removed from the population when selected
 - ▶ The same item may be picked more than once
 - ▶ Can be used to increase/decrease data volume and generate different data sets to train classification models
- ⌕ **Stratified sampling** : partition data and random items from each partition
 - ▶ In RapidMiner, partitions are specified by class labels. Class distribution in sampled data set $\approx$ original class distribution

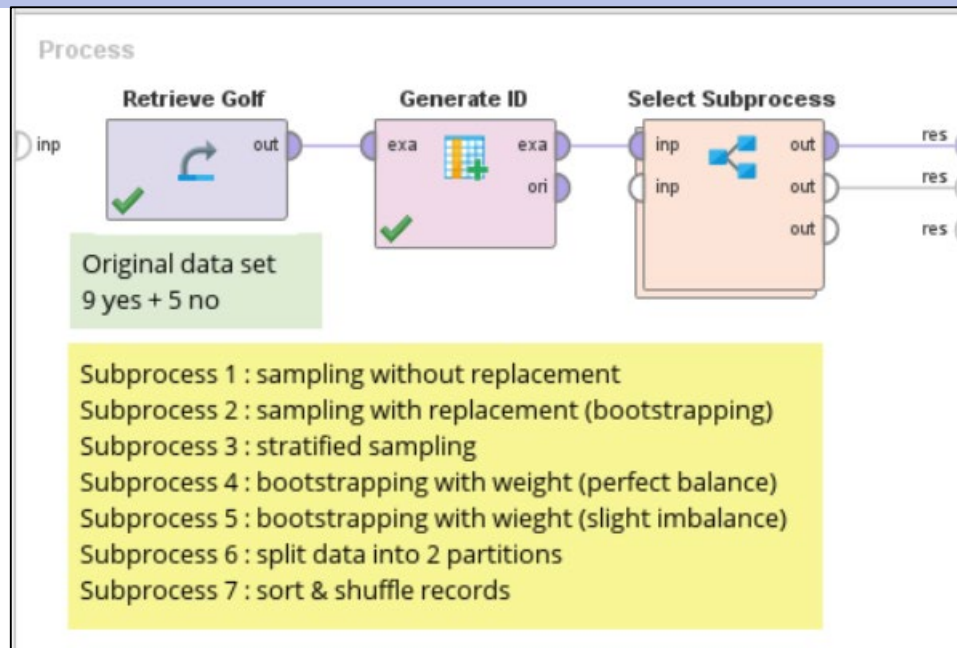
⊕ Sampling with bias (supervised sampling)

- ▶ Sampling is influenced by class labels. Items that belong to different classes have unequal chances to be picked
- ▶ Possible bias
 - Data distribution : weight toward uniform distribution (e.g. if most population = male, then sample female more frequently)
 - Frequency ratio : between the most common class and the rarest class (e.g. keep the ratio between male : female = 3:1)
- ▶ RapidMiner doesn't support supervised sampling directly
 - Assign different weights to different records
 - Then use sampling with replacement + weight

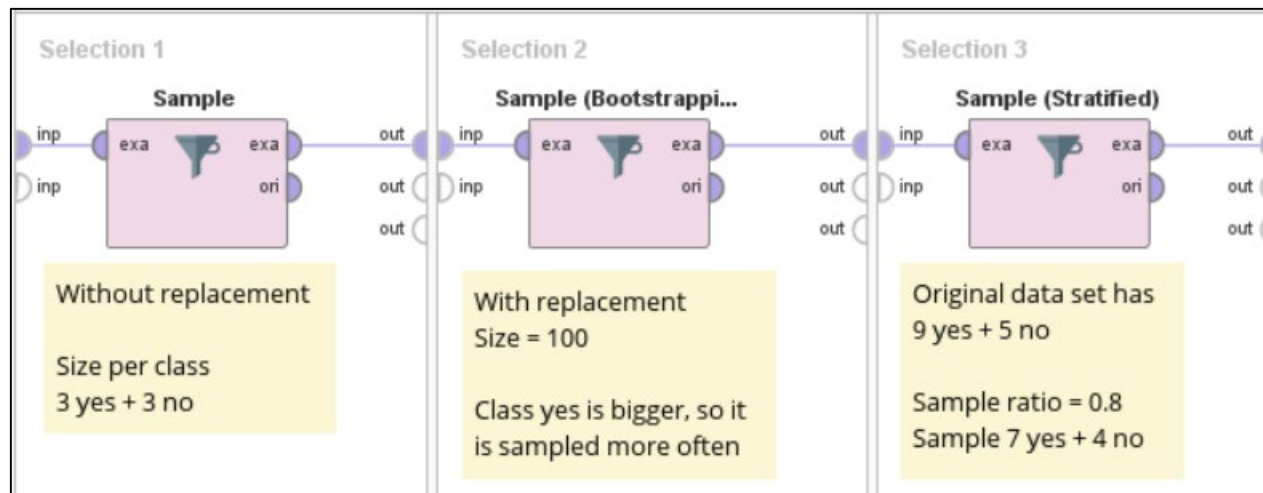
Record Manipulation in RapidMiner

Sample	Sampling without replacement
Sample (bootstrapping)	Sampling with replacement
Sample (stratified)	Stratified sampling (without replacement)
Split	Split data into subsets (sampling records in each subset)
Shuffle	Shuffle records
Sort	Sort records according to specified attribute
Filter examples	Keep records by conditions
Filter example range	Keep records whose indices are in given range

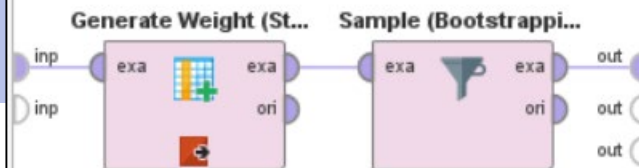
Example (6) : record manipulation



Folder
Blending.Examples



Selection 4

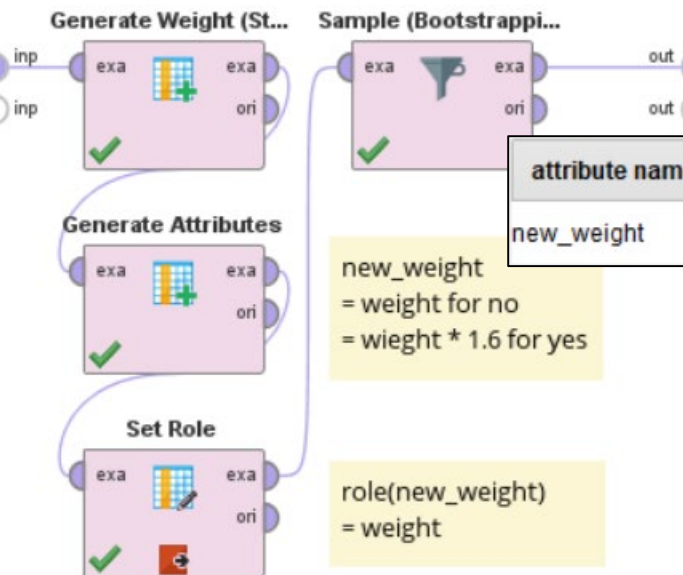


Assign weights to all records such that total weight for both classes are equal & grand total weight = given param

Total weight for no = $10 * 5 = 50$
 Total weight for yes = $5.556 * 9 = 50.004$
 Total weight = 100.004

In bootstrapping with weights, records with high weights will be sampled more often → balance data distribution

Selection 5



new_weight
 = weight for no
 = weight * 1.6 for yes

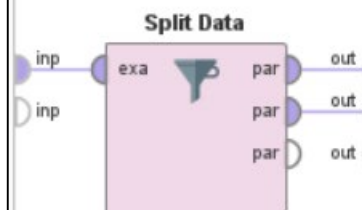
role(new_weight)
 = weight

attribute name	function expressions
new_weight	if(Play=="no", weight, weight*1.6)

For supervised sampling,
 we have to assign weights
 to individual records

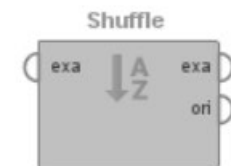
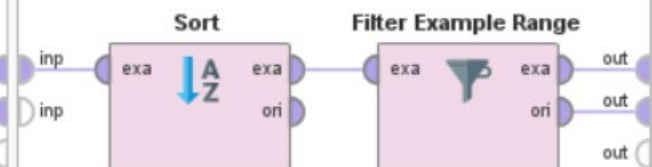
Compare results from
 subprocesses 2, 4, 5

Selection 6



Split data into 2
 partitions, with
 ratios 0.7 + 0.3

Selection 7



Sort or shuffle records. Then keep the first 4 records in the resulting table

Table Manipulation in RapidMiner

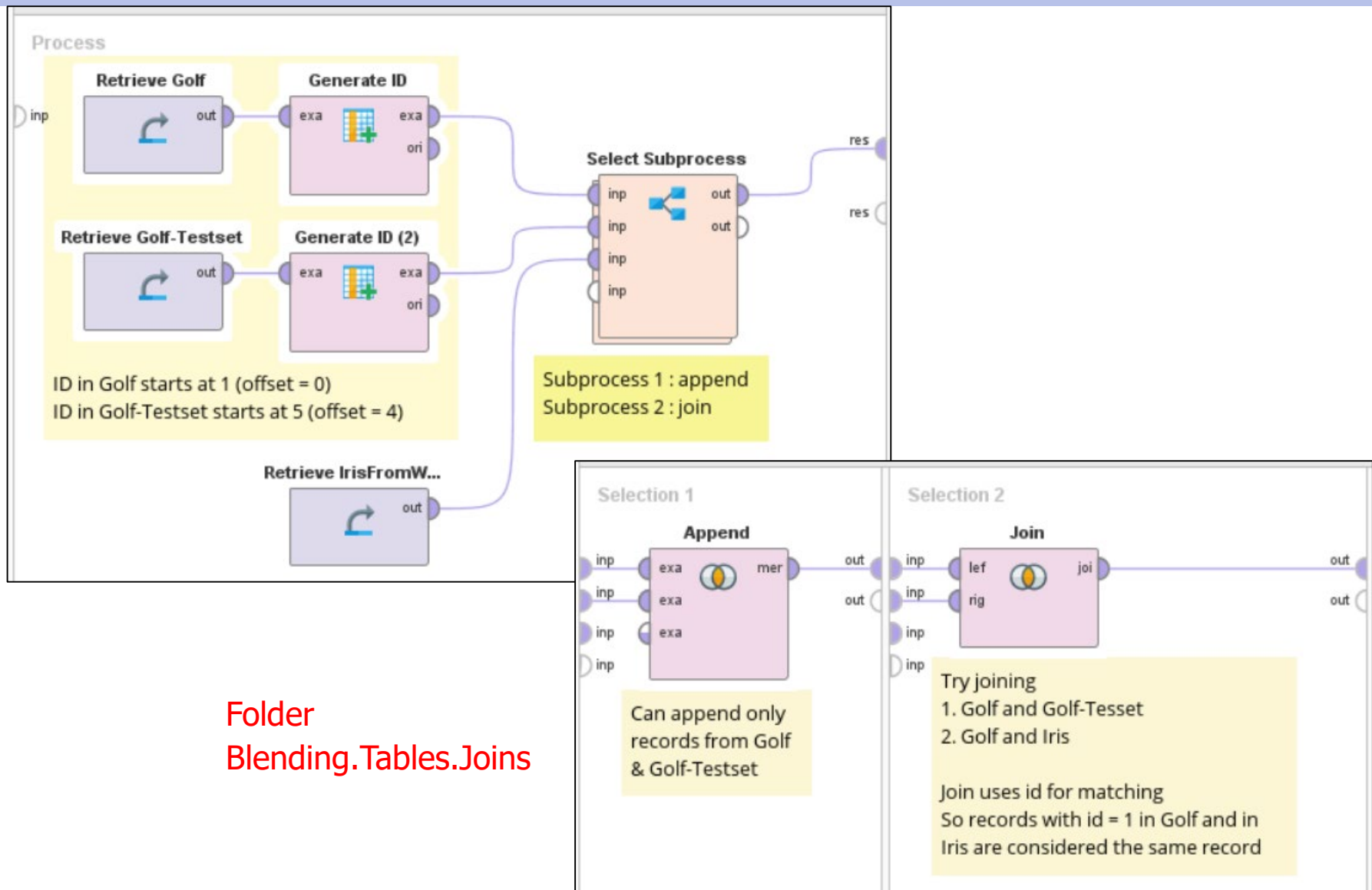
Multiple tables with the same structure (i.e. attributes)

Append	New table contains records from all tables
--------	--

2 tables (left and right) with any structure

Join	Use attribute existing in both tables as key to identify matching Inner join keep only matching records Left join keep all records from left table Right join keep all records from right table Outer join keep all records from both tables In case of left, right, outer join : if a record has no match in the other table, attributes from the other table will have missing values
------	---

Example (7) : table manipulation



Folder
Blending.Tables.Joins

Feature Selection & Aggregation

- ⊕ Identify important attributes for modeling & remove useless ones
 - ▶ Better visualization, better results
 - ▶ Computational efficiency : time & memory required
- ⊕ Attributes that can be filtered out
 - ▶ Correlated attributes, e.g. product price and VAT
 - ▶ Irrelevant to data analysis task
 - ▶ Very low variance or SD
 - Almost the same value for every record
 - $SD = 0$ indicates no variation
 - ▶ Very high variance or SD
 - All values are unique

Approaches

⊕ Attribute scoring/ranking

- ▶ Calculate the scores of all attributes by using some criteria
- ▶ Select attributes with highest scores

⊕ Subset selection

- ▶ Select subset of attributes that gives as good as or better result than the full set of attributes
- ▶ Search-based approach

⊕ Principal component analysis (PCA)

- ▶ Create new attributes that are combinations of old attributes
- ▶ Select fewer new attributes that sufficiently capture the variation in the original data
- ▶ Mathematical-based approach

Attribute Scoring

- ⊕ Measure the relevancy of each attribute w.r.t. class label
- ⊕ **Based on correlation** between each attribute and class
 - ▶ **Chi-square** for nominal attributes
 - ▶ **Correlation (Pearson)** for numeric attributes
- ⊕ **Based on impurity measurement** when using each attribute to split data into correct classes
 - ▶ **Gini (Generalized Inequality Index)** : similar to entropy
i.e. purer bin $\rightarrow$ lower entropy and Gini $\rightarrow$ higher score
 - ▶ **Information gain** = entropy before split – entropy after split
 - ▶ **Gain ratio** = information gain / split penalty
Split that results in small partitions $\rightarrow$ high penalty

⊕ **Based on impurity measurement** (cont.)

- ▶ **Symmetrical uncertainty** : entropy-based measurement of the mutual dependence between each attribute and class

⊕ **Other partition-based methods**

should be low

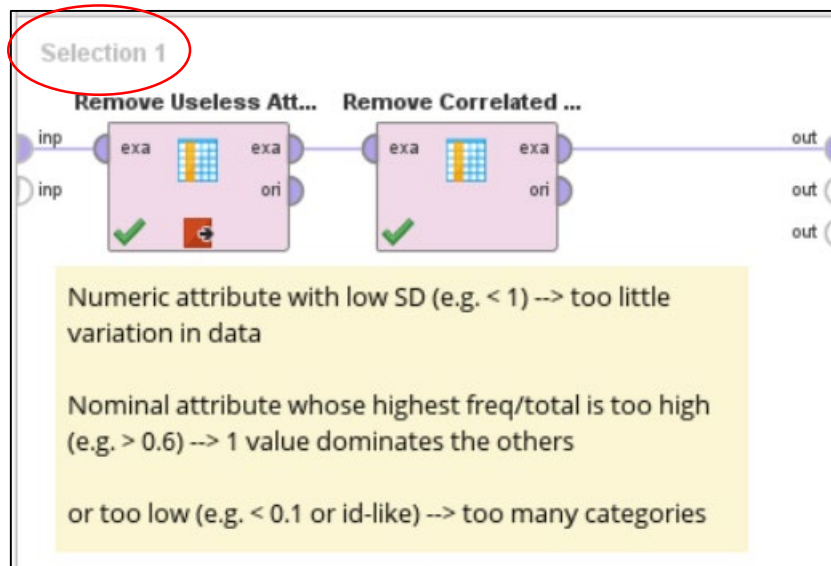
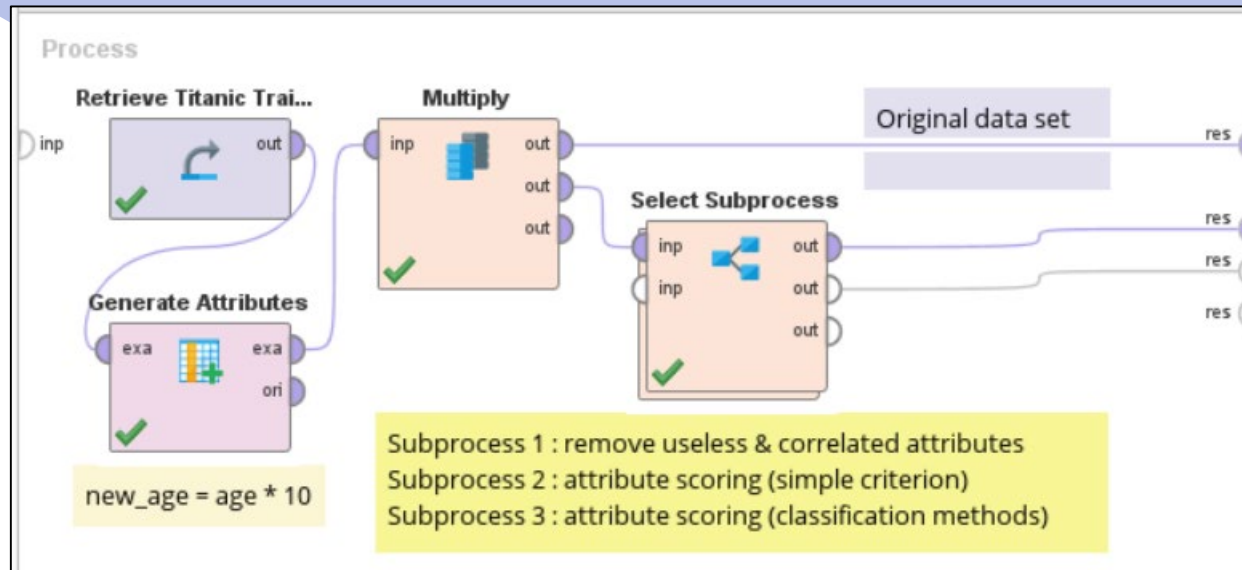
- ▶ **Relief** : compare probabilities that nearest records in the same class and in different classes have different attribute values

should be high

⊕ **By-products of classification methods**

- ▶ **OneR** : accuracy when using each attribute as a predictor in OneR rule
- ▶ **Random forest** : benefit when using each attribute as tree nodes in random forest (an ensemble of multiple decision trees)
- ▶ **SVM** : attribute weights from support vector machine (SVM)

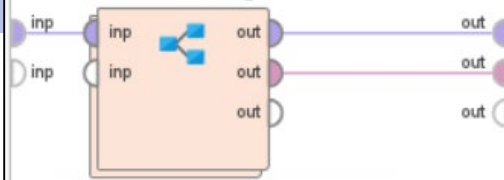
Example (8) : attribute scoring



Folders
Blending.Attributes.Selection

Selection 2

Select Scoring



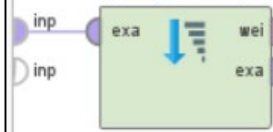
Scoring & Top-3 selection
All require class label

Subprocess 1 : chi-square
Subprocess 2 : correlation

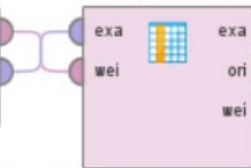
Subprocess 3 : entropy-based
Subprocess 4 : relief

Selection 1

Weight by Chi Squar...



Select by chi-square



Support only nominal attributes
Numeric attributes are discretized internally

Selection 2

Nominal to Numerical

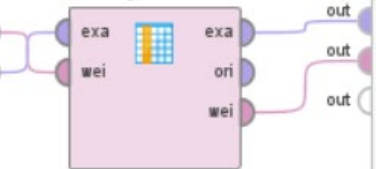


Support only
numeric and explicit
binomial attributes
--> convert nominal
attributes to
numeric first

Weight by Correlation



Select by correlation



Selection 3

Weight by Gini



Select top 3



Weight by Info Gain



Weight by Uncertainty



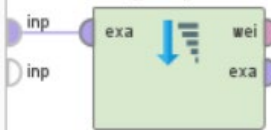
Weight by Gain Ratio



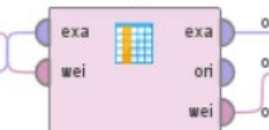
Numeric
attributes are
discretized
internally

Selection 4

Weight by Relief



Select by relief



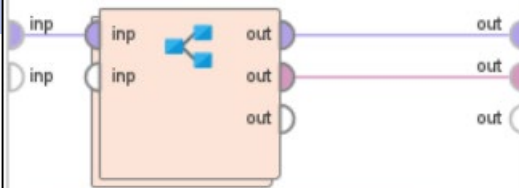
Require
#neighbors

Attributes that are very
helpful in prediction
are always chosen by
many different methods

Folders
Modeling.Feature Weights
Blending.Attributes.Selection

Selection 3

Select Classifier

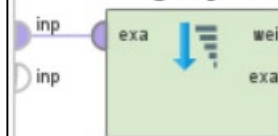


Scoring by classifier & Top-3 selection

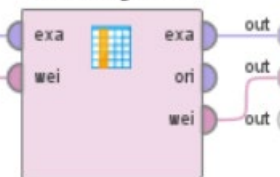
Subprocess 1 : rule
Subprocess 2 : random forest
Subprocess 3 : SVM

Selection 1

Weight by Rule

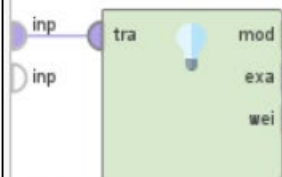


Select by rule



Selection 2

Random Forest

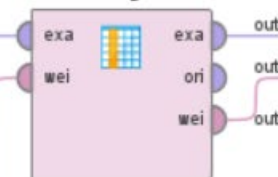


Build a random forest of 10 decision trees & use trees to evaluate attributes

Weight by Tree Imp...



Select by tree



Selection 3

Nominal to Numeric...

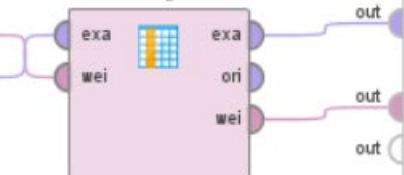


SVM works with numeric and explicit binomial attributes
--> convert all nominal attributes to numeric first

Weight by SVM



Select by SVM



Subset Selection

- ⊕ Iteratively choose an attribute to add to or remove from a subset
 - ▶ To improve classification performance → higher accuracy, or the same accuracy but with fewer attributes
 - ▶ Use a simple classifier to check the result

⊕ Forward selection

- ▶ From empty set, keep adding attributes one-by-one
- ▶ Hard to recognize interaction between attributes

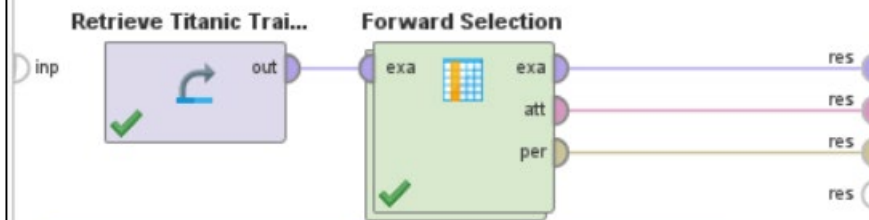
⊕ Backward elimination

- ▶ From complete set, keep removing attributes one-by-one
- ▶ May suffer from overfitting

Forward selection	Backward elimination
Initial attribute set: $\{A_1, A_2, A_3, A_4, A_5, A_6\}$	Initial attribute set: $\{A_1, A_2, A_3, A_4, A_5, A_6\}$
Initial reduced set: $\{\}$	$\Rightarrow \{A_1, A_3, A_4, A_5, A_6\}$
$\Rightarrow \{A_1\}$	$\Rightarrow \{A_1, A_4, A_5, A_6\}$
$\Rightarrow \{A_1, A_4\}$	$\Rightarrow$ Reduced attribute set: $\{A_1, A_4, A_6\}$
$\Rightarrow$ Reduced attribute set: $\{A_1, A_4, A_6\}$	

Example (9) : forward subset selection

Folder Modeling.Optimization.Feature Selection

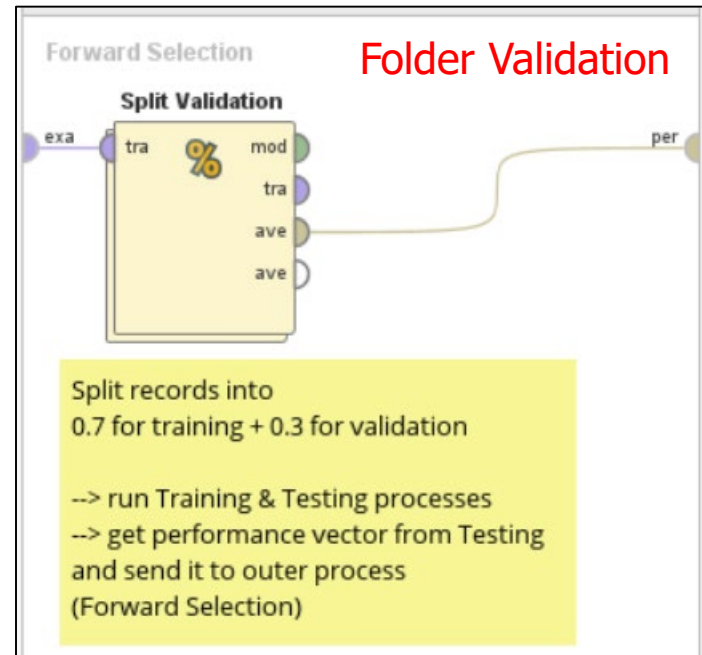


Stopping behavior = without increase (Forward)
 Stopping behavior = with decrease (Backward)

Inside subprocess : run performance evaluation that returns performance vector based on the selected attributes (port per)

Other returned values
 Attribute weights (port att)
 Reduced data set (port exa)

Folder Validation

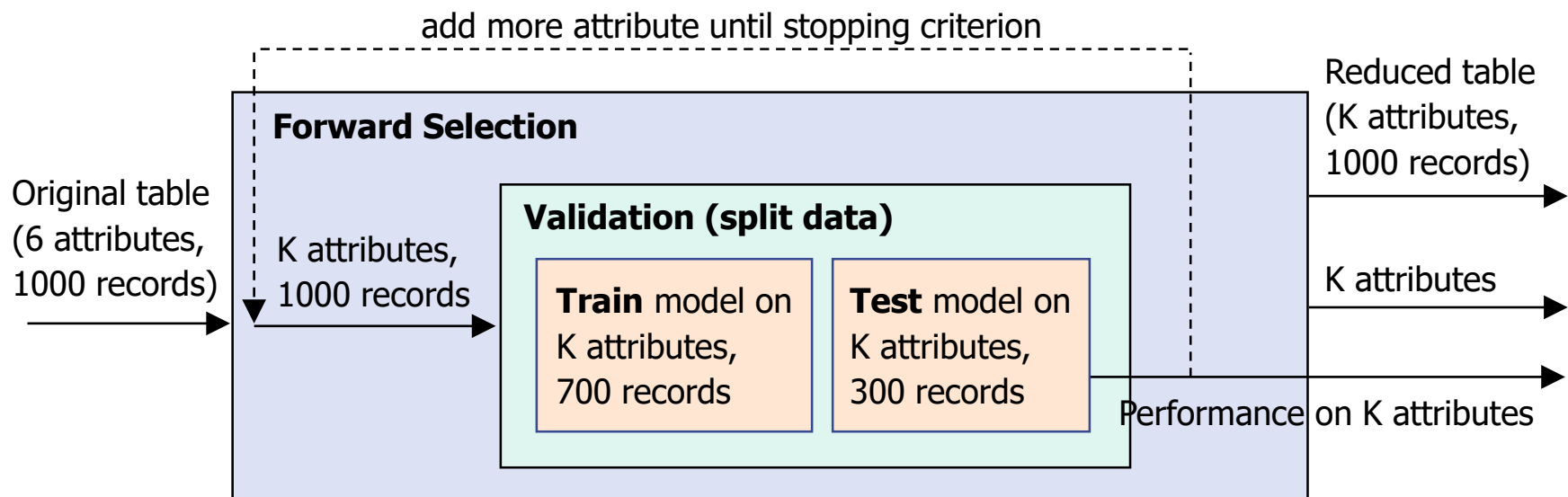
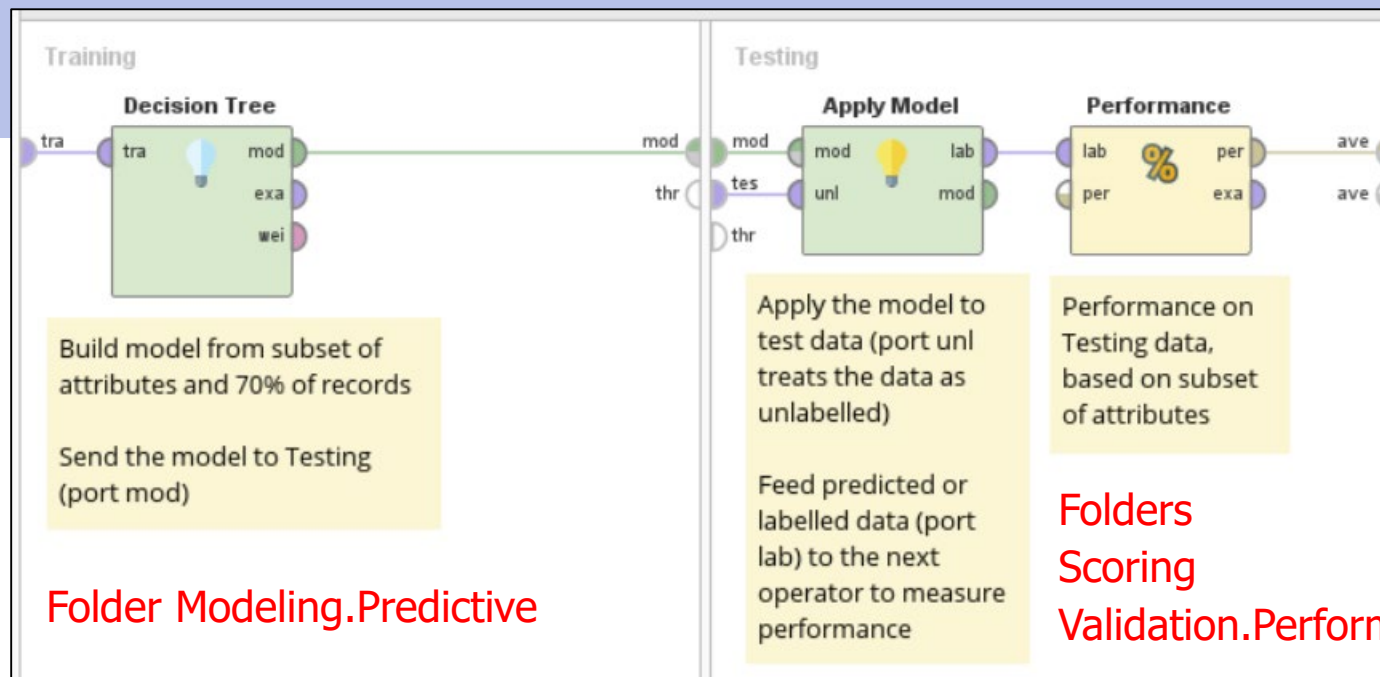


Split records into
 0.7 for training + 0.3 for validation

--> run Training & Testing processes
 --> get performance vector from Testing and send it to outer process (Forward Selection)

Try replacing "Forward Selection" with "Backward Elimination" or "Optimize Selection" with forward or backward searching

Backward method tends to keep more attributes than forward method



Attribute & Value aggregation

⊕ Purposes

- ▶ Reduce the number of attributes → computational efficiency
- ▶ Change scale → data become more stable & less variability
- ▶ Privacy protection

	Rank	Responsibility	Department	Guess who ?
1	Asst. Prof.	Head of Dept	EGCO	1 candidate
2	Asst. Prof.	Head of Dept	SCMA	1 candidate
3	Asst. Prof.	Teaching	EGCO	3 candidates
4	Assoc. Prof.	Teaching	EGCO	2 candidates
5	Prof.	Teaching	SCPY	2 candidates
6	Officer	Administrative	EGEE	3 candidates
7	Officer	Technician	EGCO	3 candidates

4 categories

4 categories

easy guess

	Role	Department	Guess who ?
1	Academic	EGCO	14 candidates
2	Academic	SCMA	43 candidates
3	Academic	EGCO	14 candidates
4	Academic	EGCO	14 candidates
5	Academic	SCPY	39 candidates
6	Support	EGEE	7 candidates
7	Support	EGCO	6 candidates

Attribute aggregation

Use Role (2 categories)
instead of Rank +
Responsibility

Still too many categories
in Department

	Role	Faculty
1	Academic	EG
2	Academic	SC
3	Academic	EG
4	Academic	EG
5	Academic	SC
6	Support	EG
7	Support	EG

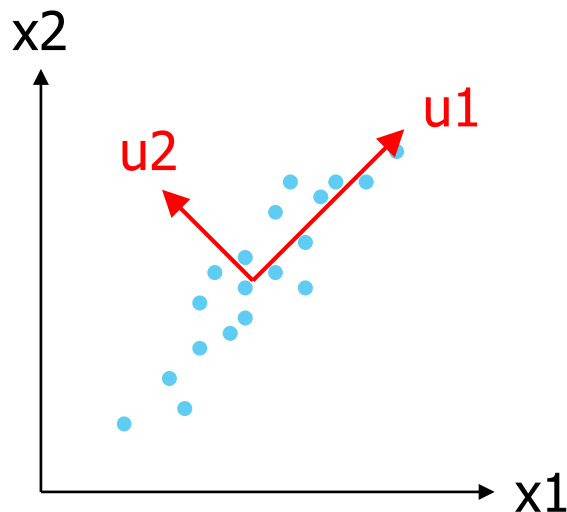
Value aggregation

Use Faculty instead of Department

Can be done by using attribute generation
& value manipulation operators

Principal Component Analysis (PCA)

- ⊕ Find new attributes, i.e. principal components, which are
 - ▶ Linear combination of original ones (numeric)
 - ▶ Capture the largest amount of variation in data
 - ▶ Independent of each other (i.e. orthogonal to each other in the original space)



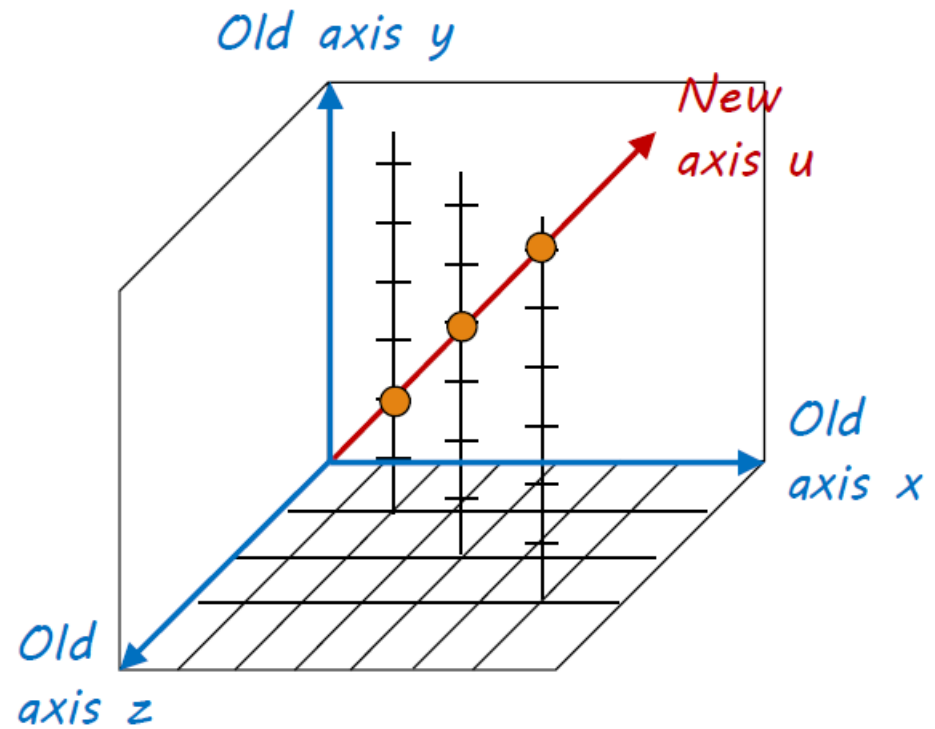
x_1 and x_2 are combined to form u_1

u_1 captures the most variation in data,
then u_2 captures remaining variation, ...

Basic Idea

Original data set			
Record	x	y	z
1	2	2	1
2	4	4	2
3	6	6	3

w.r.t new axis	
Record	u
1	1
2	2
3	3



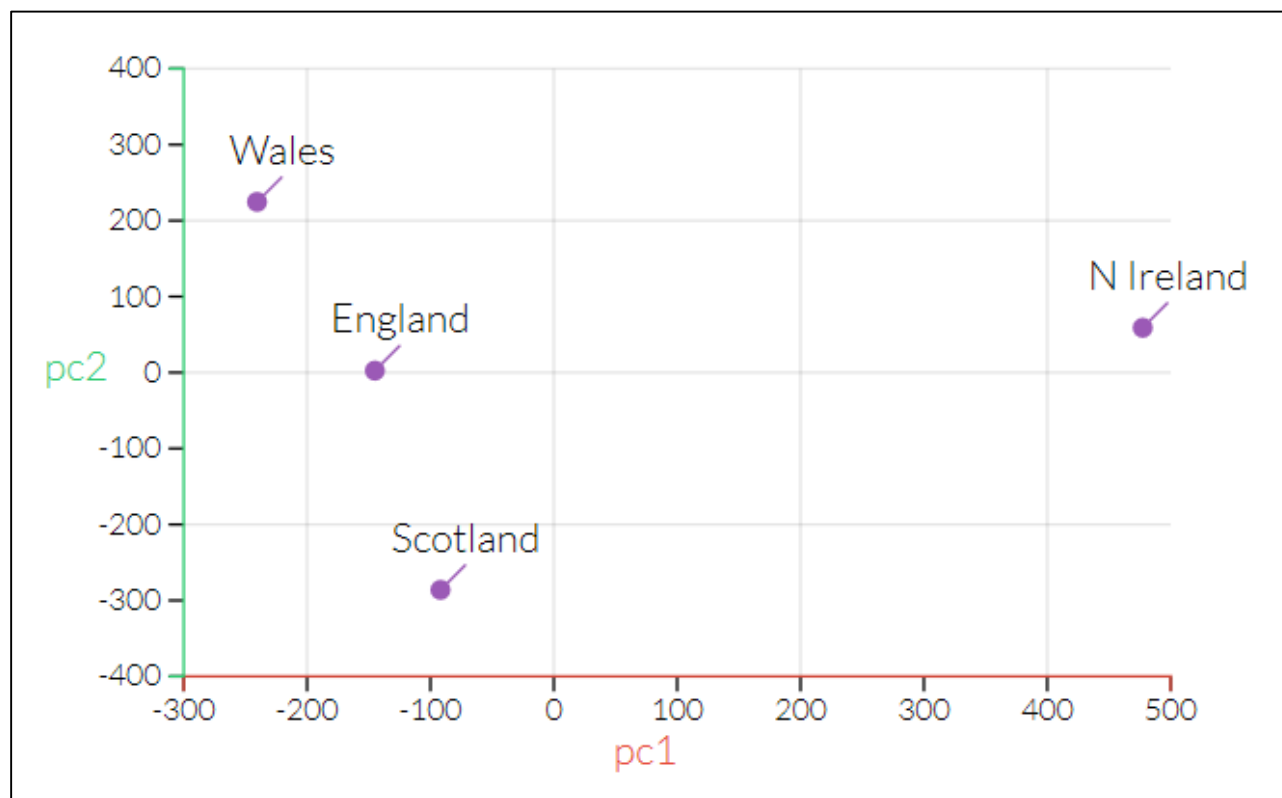
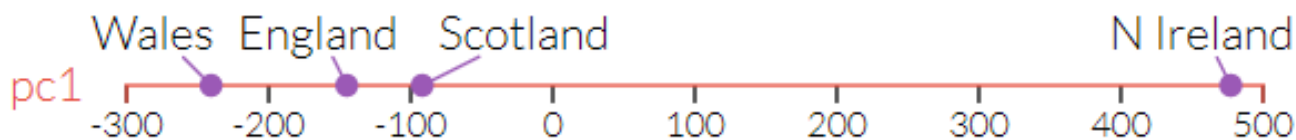
PCA Visualization

<http://setosa.io/ev/principal-component-analysis/>

	England	N Ireland	Scotland	Wales
Alcoholic drinks	375	135	458	475
Beverages	57	47	53	73
Carcase meat	245	267	242	227
Cereals	1472	1494	1462	1582
Cheese	105	66	103	103
Confectionery	54	41	62	64
Fats and oils	193	209	184	235
Fish	147	93	122	160
Fresh fruit	1102	674	957	1137
Fresh potatoes	720	1033	566	874
Fresh Veg	253	143	171	265
Other meat	685	586	750	803
Other Veg	488	355	418	570
Processed potatoes	198	187	220	203
Processed Veg	360	334	337	365
Soft drinks	1374	1506	1572	1256
Sugars	156	139	147	175

Average consumption
of 17 types of food
(grams/person/week)
in 4 regions of UK

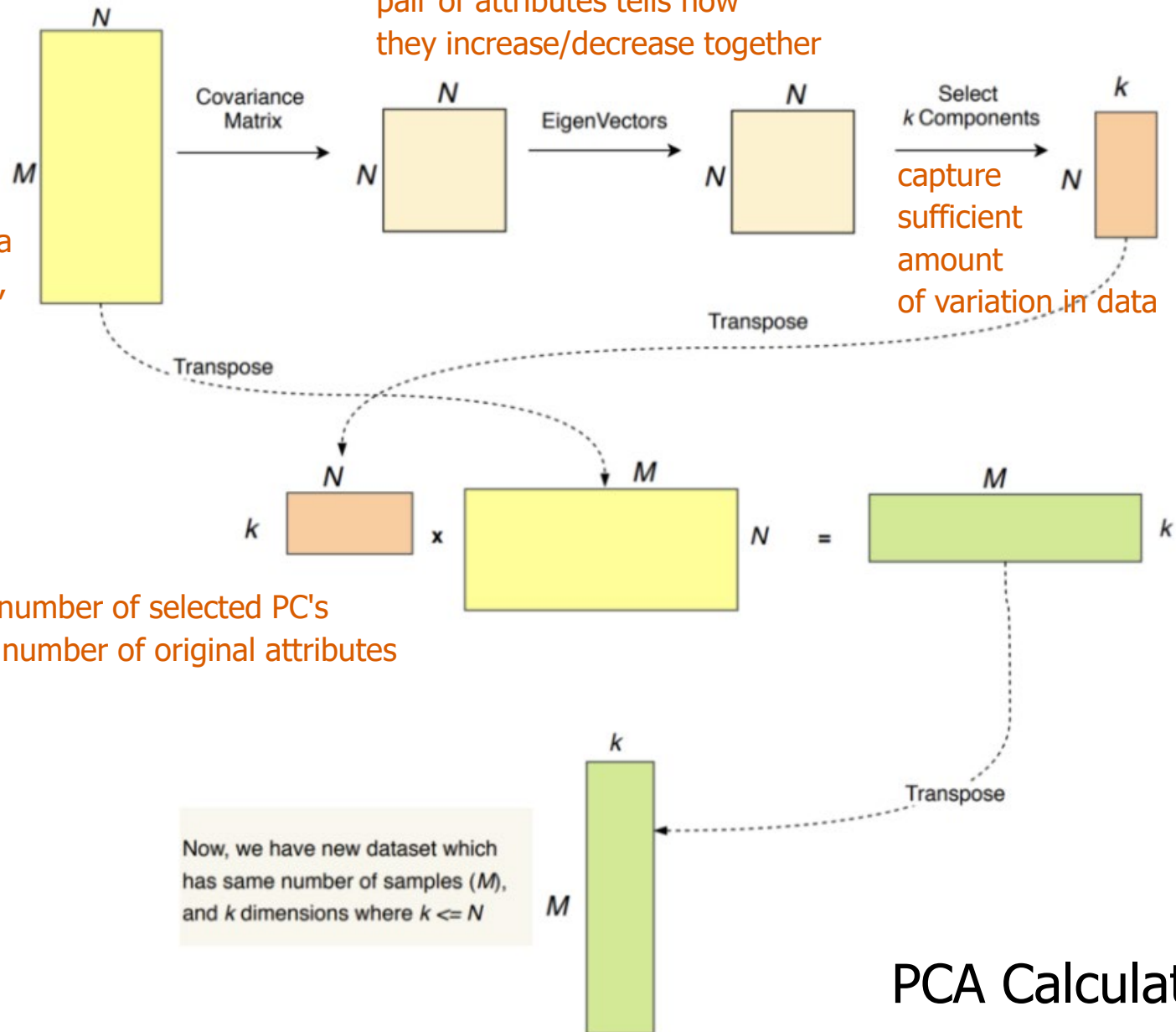
With 17 attributes,
difference in overall
food consumption in
different regions is
not clear



From just pc1 or pc1-pc2 components, we can see that food consumption in N.Ireland is clearly different from others

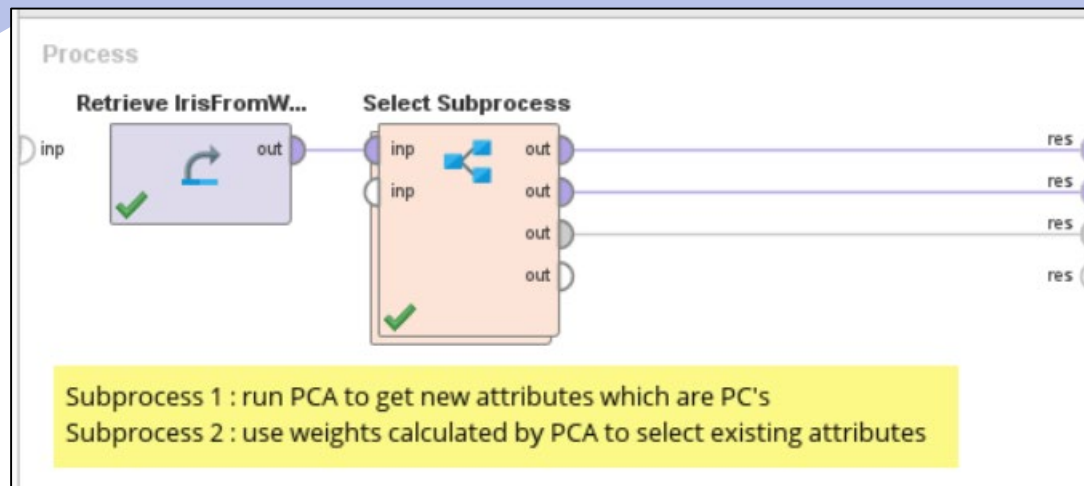
Covariance between each pair of attributes tells how they increase/decrease together

Original data
N attributes,
M records

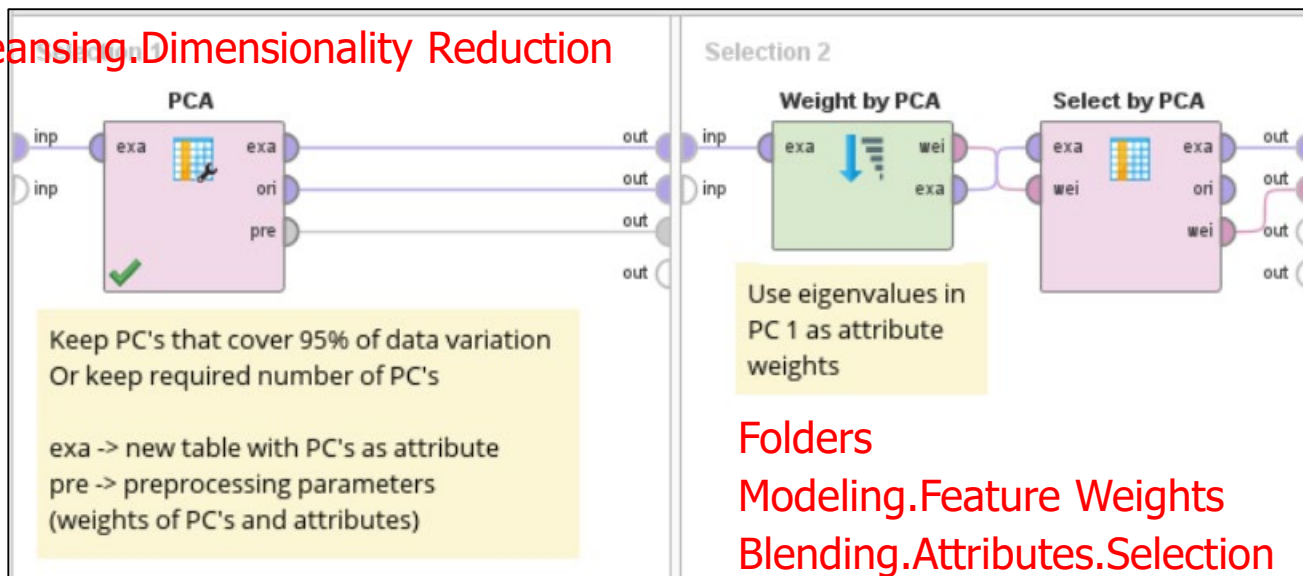


PCA Calculation

Example (10) : PCA



Folder Cleansing.Dimensionality Reduction



Row No.	id	label	sepal length	sepal width	petal length	petal width
1	id_1	Iris-setosa	5.100	3.500	1.400	0.200
2	id_2	Iris-setosa	4.900	3	1.400	0.200
3	id_3	Iris-setosa	4.700	3.200	1.300	0.200
4	id_4	Iris-setosa	4.600	3.100	1.500	0.200
5	id_5	Iris-setosa	5	3.600	1.400	0.200

Original
data set

Row No.	id	label	pc_1	pc_2
1	id_1	Iris-setosa	-2.684	-0.327
2	id_2	Iris-setosa	-2.715	0.170
3	id_3	Iris-setosa	-2.890	0.137
4	id_4	Iris-setosa	-2.746	0.311
5	id_5	Iris-setosa	-2.729	-0.334

New data set (subprocess 1)

Row No.	id	class	sepal length	petal length
1	1	Iris-setosa	5.100	1.400
2	2	Iris-setosa	4.900	1.400
3	3	Iris-setosa	4.700	1.300
4	4	Iris-setosa	4.600	1.500
5	5	Iris-setosa	5	1.400

New data set (subprocess 2)

 Eigenvalues	Summarize properties of each PC			
	Component	Standard Deviation	Proportion of Variance	Cumulative Variance
 Eigenvectors	PC 1	2.055	0.925	0.925
	PC 2	0.492	0.053	0.978
	PC 3	0.280	0.017	0.995
	PC 4	0.154	0.005	1.000

PC1 covers 92.5% of variation in data already

PC1 + PC2 are chosen because our threshold = 0.95

 Eigenvalues					
	Attribute	PC 1	PC 2	PC 3	PC 4
 Eigenvectors	sepal length	0.362	-0.657	-0.581	-0.317
	sepal width	-0.082	-0.730	0.596	0.324
	petal length	0.857	0.176	0.073	0.480
	petal width	0.359	0.075	0.549	-0.751

To select original attributes, use eigenvalues in PC1 as attribute weights

Eigenvectors (= principal components) determine direction (axis) of the new feature space, with eigenvalues as magnitudes